

Chapter 46 — Trie

0. What You Will Learn in This Chapter

A **Trie**, also called a **prefix tree**, is a tree data structure used to store strings by shared prefixes.

Trie is one of the most important advanced data structures for string problems.

It is especially useful when you need to:

- insert words
- search words
- search prefixes
- autocomplete
- find dictionary words
- support wildcard search
- search many words in a grid
- replace words by shortest root
- compute maximum XOR using binary representation
- handle prefix/suffix queries
- optimize many repeated string lookups

By the end of this chapter, you should understand:

- What a Trie is
- Why Trie is useful
- How Trie nodes work
- How to implement insert, search, and startsWith
- How to handle wildcard search
- How to combine Trie with DFS/backtracking
- How to solve Word Search II
- How to use Trie for autocomplete
- How to use Trie for root replacement
- How to use binary Trie for maximum XOR
- How to design prefix/suffix search
- How to analyze Trie memory and time complexity

This is a hard-core chapter, so it includes more explanations and examples than a standard chapter.

Core problems covered in this chapter:

1. Implement Trie
2. Design Add and Search Words Data Structure
3. Word Search II
4. Replace Words
5. Search Suggestions System
6. Map Sum Pairs
7. Maximum XOR of Two Numbers in an Array
8. Prefix and Suffix Search
9. Stream of Characters
10. Longest Word in Dictionary
11. Short Encoding of Words
12. Count Prefix and Suffix Pairs, preview
13. Trie memory optimization
14. Binary Trie

1. What Is a Trie?

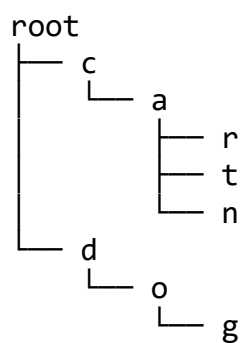
A Trie is a tree where each edge represents a character.

Words with common prefixes share the same path.

Example words:

car
cat
can
dog

Trie structure:



The prefix:

ca

is shared by:

car, cat, can

This is the main advantage of Trie.

2. Why Trie Exists

Suppose you have many words and need many prefix queries.

With a hash set, exact word search is easy:

$O(\text{length})$

But prefix search is harder unless you store all prefixes.

With Trie:

insert word	$O(\text{length})$
search word	$O(\text{length})$
startsWith	$O(\text{length})$

This is ideal for dictionaries and autocomplete.

3. Trie vs Hash Set

Operation	Hash Set	Trie
Exact word search	$O(\text{length})$ average	$O(\text{length})$
Prefix search	not direct	$O(\text{prefix length})$
Autocomplete	inefficient	natural
Wildcard search	difficult	DFS over Trie
Memory	often lower	can be high
Shared prefixes	no	yes

Use hash set for exact lookup.

Use Trie when prefix structure matters.

4. Trie Node Structure

For lowercase English letters, each node can have 26 children.

```
struct TrieNode {
    TrieNode* children[26];
    bool isEnd;

    TrieNode() {
        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }

        isEnd = false;
    }
};
```

`isEnd` marks whether a full word ends at this node.

Example:

If words are:

```
app
apple
```

The node for `p` in `app` has:

```
isEnd = true
```

and still has child path to `apple`.

5. When to Use Trie

Use Trie when the problem involves:

5.1 Prefix Search

Examples:

- Implement Trie

- Search Suggestions System
- Longest Word in Dictionary

5.2 Dictionary Word Matching

Examples:

- Word Search II
- Replace Words

5.3 Wildcard Matching

Example:

- Design Add and Search Words Data Structure

5.4 Many String Queries

If you repeatedly search many words/prefixes, Trie can reduce repeated work.

5.5 Binary Prefixes

Binary Trie is useful for XOR problems.

Example:

- Maximum XOR of Two Numbers
-

6. When Not to Use Trie

Do not use Trie when:

6.1 Only Exact Word Lookup Is Needed

Use hash set.

6.2 Number of Words Is Small

Simple methods may be easier.

6.3 Prefix Does Not Matter

Trie may be overkill.

6.4 Memory Is Very Tight

Trie nodes can consume significant memory.

6.5 You Need Substring Search

Use KMP, rolling hash, suffix automaton, or suffix array depending on problem.

Trie is naturally for prefixes, not arbitrary substrings.

7. Mental Models

7.1 Trie Is a Shared Prefix Tree

Every root-to-node path represents a prefix.

7.2 End Marker Matters

A path can exist without being a complete word.

Example:

If Trie contains:

apple

Then prefix:

app

exists, but word app exists only if isEnd is true.

7.3 Search Is Just Walking

For each character:

```
move to child
if child missing, fail
```

7.4 Wildcard Means Branching

If character is ., try all children.

7.5 Grid + Trie Means Pruned Backtracking

In Word Search II, Trie tells us whether current prefix can still lead to a word.

If not, stop early.

8. Pattern Recognition Checklist

Ask:

16. Do I need prefix lookup?
 17. Do I need autocomplete suggestions?
 18. Are there many dictionary words?
 19. Am I searching many words in a grid?
 20. Does the problem involve wildcard character matching?
 21. Do I need shortest root prefix?
 22. Do I need maximum XOR?
 23. Can strings share prefixes?
 24. Would hash set require checking many prefixes repeatedly?
 25. Is this a prefix problem or substring problem?
-

9. C++ Tools Needed

9.1 Trie Node With Array

```
struct TrieNode {
    TrieNode* children[26];
    bool isEnd;

    TrieNode() {
        memset(children, 0, sizeof(children));
        isEnd = false;
    }
};
```

Fast and common for lowercase letters.

9.2 Trie Node With unordered_map

```
struct TrieNode {
    unordered_map<char, TrieNode*> children;
    bool isEnd = false;
};
```

Use when alphabet is large or unknown.

9.3 Storing Word at End Node

For Word Search II:

```
string word;
```

inside node can help avoid reconstructing path.

10. Core Template 1 — Basic Trie Class

```
class Trie {
private:
    struct TrieNode {
        TrieNode* children[26];
        bool isEnd;

        TrieNode() {
            memset(children, 0, sizeof(children));
            isEnd = false;
        }
    };

    TrieNode* root;

public:
    Trie() {
        root = new TrieNode();
    }

    void insert(string word) {
        TrieNode* node = root;

        for (char c : word) {
            int index = c - 'a';

            if (!node->children[index]) {
                node->children[index] = new TrieNode();
            }

            node = node->children[index];
        }

        node->isEnd = true;
    }

    bool search(string word) {
        TrieNode* node = root;

        for (char c : word) {
```

```

        int index = c - 'a';

        if (!node->children[index]) {
            return false;
        }

        node = node->children[index];
    }

    return node->isEnd;
}

bool startsWith(string prefix) {
    TrieNode* node = root;

    for (char c : prefix) {
        int index = c - 'a';

        if (!node->children[index]) {
            return false;
        }

        node = node->children[index];
    }

    return true;
}
};

```

11. Core Template 2 — Trie With Word Storage

Useful for returning found words.

```

struct TrieNode {
    TrieNode* children[26];
    string word;

    TrieNode() {
        memset(children, 0, sizeof(children));
        word = "";
    }
};

```

Insert:

```

void insert(string word) {
    TrieNode* node = root;

```

```

    for (char c : word) {
        int index = c - 'a';

        if (!node->children[index]) {
            node->children[index] = new TrieNode();
        }

        node = node->children[index];
    }

    node->word = word;
}

```

If `node->word != ""`, a word ends there.

12. Core Template 3 — Wildcard Trie DFS

For pattern containing `.`:

```

bool dfs(string& word, int pos, TrieNode* node) {
    if (!node) {
        return false;
    }

    if (pos == word.size()) {
        return node->isEnd;
    }

    char c = word[pos];

    if (c == '.') {
        for (int i = 0; i < 26; i++) {
            if (dfs(word, pos + 1, node->children[i])) {
                return true;
            }
        }

        return false;
    }

    return dfs(word, pos + 1, node->children[c - 'a']);
}

```

13. Core Template 4 — Binary Trie

For XOR problems, use bits instead of characters.


```

struct Node {
    Node* child[2];

    Node() {
        child[0] = nullptr;
        child[1] = nullptr;
    }
};

```

Insert number:

```

void insert(int num) {
    Node* node = root;

    for (int bit = 31; bit >= 0; bit--) {
        int b = (num >> bit) & 1;

        if (!node->child[b]) {
            node->child[b] = new Node();
        }

        node = node->child[b];
    }
}

```

To maximize XOR, try opposite bit first.

14. Full Problem Walkthrough 1 — LeetCode 208: Implement Trie

14.1 Problem Summary

Implement:

```

insert(word)
search(word)
startsWith(prefix)

```

14.2 C++ Code

```

class Trie {
private:
    struct TrieNode {
        TrieNode* children[26];
        bool isEnd;

        TrieNode() {
            memset(children, 0, sizeof(children));
            isEnd = false;
        }
    };
};

```

```

TrieNode* root;

public:
    Trie() {
        root = new TrieNode();
    }

    void insert(string word) {
        TrieNode* node = root;

        for (char c : word) {
            int index = c - 'a';

            if (!node->children[index]) {
                node->children[index] = new TrieNode();
            }

            node = node->children[index];
        }

        node->isEnd = true;
    }

    bool search(string word) {
        TrieNode* node = root;

        for (char c : word) {
            int index = c - 'a';

            if (!node->children[index]) {
                return false;
            }

            node = node->children[index];
        }

        return node->isEnd;
    }

    bool startsWith(string prefix) {
        TrieNode* node = root;

        for (char c : prefix) {
            int index = c - 'a';

            if (!node->children[index]) {
                return false;
            }

```

```

        node = node->children[index];
    }

    return true;
}
};

```

14.3 Complexity

For word length L:

```

insert:    O(L)
search:    O(L)
startsWith: O(L)

```

Space:

$O(\text{total characters inserted} * \text{alphabet size overhead})$

15. Full Problem Walkthrough 2 — LeetCode 211: Design Add and Search Words Data Structure

15.1 Problem Summary

Support:

```

addWord(word)
search(word)

```

Search word may contain wildcard . that matches any letter.

Example:

```

bad
dad
mad

```

Search:

```

.pad -> false
bad  -> true
.ad  -> true
b..  -> true

```

15.2 Pattern

Trie + DFS branching on wildcard.

15.3 C++ Code

```

class WordDictionary {
private:
    struct TrieNode {
        TrieNode* children[26];
    };

```

```

        bool isEnd;

        TrieNode() {
            memset(children, 0, sizeof(children));
            isEnd = false;
        }
    };

    TrieNode* root;

    bool dfs(string& word, int pos, TrieNode* node) {
        if (!node) {
            return false;
        }

        if (pos == word.size()) {
            return node->isEnd;
        }

        char c = word[pos];

        if (c == '.') {
            for (int i = 0; i < 26; i++) {
                if (dfs(word, pos + 1, node->children[i])) {
                    return true;
                }
            }

            return false;
        }

        return dfs(word, pos + 1, node->children[c - 'a']);
    }

public:
    WordDictionary() {
        root = new TrieNode();
    }

    void addWord(string word) {
        TrieNode* node = root;

        for (char c : word) {
            int index = c - 'a';

            if (!node->children[index]) {
                node->children[index] = new TrieNode();
            }
        }
    }

```

```

        node = node->children[index];
    }

    node->isEnd = true;
}

bool search(string word) {
    return dfs(word, 0, root);
}
};

```

15.4 Complexity

Without wildcards:

$O(L)$

With many wildcards:

Worst-case $O(26^L)$

But pruning by Trie structure helps in practice.

16. Full Problem Walkthrough 3 — LeetCode 212: Word Search II

16.1 Problem Summary

Given a 2D board of characters and a list of words, return all words found in the board. Words can be formed by moving:

up, down, left, right

A cell can be used only once in a word.

16.2 Why Trie Is Needed

Naive:

For each word, run DFS on board.

If there are many words, this is expensive.

Trie approach:

- Insert all words into Trie.
- DFS from every board cell.
- Stop immediately when current path is not a prefix in Trie.

This prunes huge search space.

16.3 State

DFS state:

row, col, TrieNode*

The current board path corresponds to the Trie node.

16.4 C++ Code

```

class Solution {
private:
    struct TrieNode {

```

```

TrieNode* children[26];
string word;

TrieNode() {
    memset(children, 0, sizeof(children));
    word = "";
}

};

TrieNode* root;
vector<string> result;
int rows;
int cols;

void insert(string word) {
    TrieNode* node = root;

    for (char c : word) {
        int index = c - 'a';

        if (!node->children[index]) {
            node->children[index] = new TrieNode();
        }

        node = node->children[index];
    }

    node->word = word;
}

void dfs(vector<vector<char>>& board, int r, int c, TrieNode*
node) {
    if (r < 0 || r >= rows || c < 0 || c >= cols) {
        return;
    }

    char ch = board[r][c];

    if (ch == '#') {
        return;
    }

    int index = ch - 'a';

    if (!node->children[index]) {
        return;
    }

    node = node->children[index];
}

```

```

        if (!node->word.empty()) {
            result.push_back(node->word);
            node->word = "";
        }

        board[r][c] = '#';

        dfs(board, r + 1, c, node);
        dfs(board, r - 1, c, node);
        dfs(board, r, c + 1, node);
        dfs(board, r, c - 1, node);

        board[r][c] = ch;
    }

public:
    vector<string> findWords(vector<vector<char>>& board,
vector<string>& words) {
        root = new TrieNode();

        for (string word : words) {
            insert(word);
        }

        rows = board.size();
        cols = board[0].size();

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                dfs(board, r, c, root);
            }
        }

        return result;
    }
};

```

16.5 Important Tricks

Avoid duplicates

After finding word:

```
node->word = "";
```

This prevents adding the same word multiple times.

Mark visited

```
board[r][c] = '#';
```

Restore after DFS.

16.6 Complexity

Worst case can still be high, but Trie pruning is powerful.

Roughly:

```
O(rows * cols * 4^maxWordLength)
```

worst-case, but much lower in practice due to prefix pruning.

17. Full Problem Walkthrough 4 — LeetCode 648: Replace Words

17.1 Problem Summary

Given dictionary roots and a sentence.

Replace each word in the sentence with the shortest root that is its prefix.

Example:

```
dictionary = ["cat","bat","rat"]  
sentence = "the cattle was rattled by the battery"
```

Output:

```
"the cat was rat by the bat"
```

17.2 Pattern

Insert roots into Trie.

For each word, walk Trie until:

- root end found
- path breaks
- word ends

17.3 C++ Code

```
class Solution {  
private:  
    struct TrieNode {  
        TrieNode* children[26];  
        bool isEnd;  
  
        TrieNode() {  
            memset(children, 0, sizeof(children));  
            isEnd = false;  
        }  
    };  
  
    TrieNode* root;  
  
    void insert(string word) {  
        TrieNode* node = root;  
  
        for (char c : word) {  
            int index = c - 'a';  
  
            if (!node->children[index]) {
```



```

        node->children[index] = new TrieNode();
    }

    node = node->children[index];
}

node->isEnd = true;
}

string findRoot(string word) {
    TrieNode* node = root;
    string prefix;

    for (char c : word) {
        int index = c - 'a';

        if (!node->children[index]) {
            return word;
        }

        prefix.push_back(c);
        node = node->children[index];

        if (node->isEnd) {
            return prefix;
        }
    }

    return word;
}

public:
    string replaceWords(vector<string>& dictionary, string sentence)
    {
        root = new TrieNode();

        for (string rootWord : dictionary) {
            insert(rootWord);
        }

        stringstream ss(sentence);
        string word;
        vector<string> words;

        while (ss >> word) {
            words.push_back(findRoot(word));
        }

        string result;

```

```

        for (int i = 0; i < words.size(); i++) {
            if (i > 0) {
                result.push_back(' ');
            }

            result += words[i];
        }

        return result;
    }
};

```

17.4 Complexity

Let total dictionary characters be D and sentence characters be S .

Time: $O(D + S)$

Space: $O(D)$

18. Full Problem Walkthrough 5 — LeetCode 1268: Search Suggestions System

18.1 Problem Summary

Given products and searchWord.

After each typed character, return up to 3 lexicographically smallest products with that prefix.

18.2 Approach Options

Option 1: Sort + Binary Search

Often simpler.

Option 2: Trie

Store suggestions at each Trie node.

We will use Trie.

18.3 Trie Idea

Sort products lexicographically.

Insert products into Trie.

At each node, store at most first 3 suggestions.

Because products are inserted sorted, first 3 are lexicographically smallest.

18.4 C++ Code

```

class Solution {
private:
    struct TrieNode {
        TrieNode* children[26];
        vector<string> suggestions;
    };

```

```

        TrieNode() {
            memset(children, 0, sizeof(children));
        }
};

TrieNode* root;

void insert(string product) {
    TrieNode* node = root;

    for (char c : product) {
        int index = c - 'a';

        if (!node->children[index]) {
            node->children[index] = new TrieNode();
        }

        node = node->children[index];

        if (node->suggestions.size() < 3) {
            node->suggestions.push_back(product);
        }
    }
}

public:
    vector<vector<string>> suggestedProducts(vector<string>&
products, string searchWord) {
        sort(products.begin(), products.end());

        root = new TrieNode();

        for (string product : products) {
            insert(product);
        }

        vector<vector<string>> result;
        TrieNode* node = root;

        for (char c : searchWord) {
            int index = c - 'a';

            if (node) {
                node = node->children[index];
            }

            if (node) {
                result.push_back(node->suggestions);
            } else {

```

```

        result.push_back({});
    }
}

return result;
}
};

```

18.5 Complexity

Sorting:

$O(n \log n * \text{averageLength})$

Insertion:

$O(\text{total product characters})$

Query:

$O(\text{searchWord length})$

19. Full Problem Walkthrough 6 — LeetCode 677: Map Sum Pairs

19.1 Problem Summary

Implement:

```

insert(key, val)
sum(prefix)

```

If key already exists, update value.

Return sum of all values of keys with given prefix.

19.2 Trie Idea

Each node stores:

sum of all values under this prefix

When updating existing key, compute delta:

$\text{delta} = \text{newValue} - \text{oldValue}$

Add delta to every node along key path.

19.3 C++ Code

```

class MapSum {
private:
    struct TrieNode {
        TrieNode* children[26];
        int sum;

        TrieNode() {
            memset(children, 0, sizeof(children));
            sum = 0;
        }
    }

```

```

};

TrieNode* root;
unordered_map<string, int> values;

public:
    MapSum() {
        root = new TrieNode();
    }

    void insert(string key, int val) {
        int oldValue = values.count(key) ? values[key] : 0;
        int delta = val - oldValue;

        values[key] = val;

        TrieNode* node = root;

        for (char c : key) {
            int index = c - 'a';

            if (!node->children[index]) {
                node->children[index] = new TrieNode();
            }

            node = node->children[index];
            node->sum += delta;
        }
    }

    int sum(string prefix) {
        TrieNode* node = root;

        for (char c : prefix) {
            int index = c - 'a';

            if (!node->children[index]) {
                return 0;
            }

            node = node->children[index];
        }

        return node->sum;
    }
};

```

19.4 Complexity

For key/prefix length L:

```
insert: O(L)
sum:    O(L)
```

20. Full Problem Walkthrough 7 — LeetCode 421: Maximum XOR of Two Numbers in an Array

20.1 Problem Summary

Given nums, find maximum value of:

```
nums[i] XOR nums[j]
```

20.2 Why Trie?

XOR is maximized when bits are opposite.

For each number, at each bit:

- if current bit is 0, prefer 1
- if current bit is 1, prefer 0

This is exactly a binary Trie.

20.3 C++ Code

```
class Solution {
private:
    struct Node {
        Node* child[2];

        Node() {
            child[0] = nullptr;
            child[1] = nullptr;
        }
    };

    Node* root;

    void insert(int num) {
        Node* node = root;

        for (int bit = 31; bit >= 0; bit--) {
            int b = (num >> bit) & 1;

            if (!node->child[b]) {
                node->child[b] = new Node();
            }

            node = node->child[b];
        }
    }
}
```

```

int getMaxXor(int num) {
    Node* node = root;
    int value = 0;

    for (int bit = 31; bit >= 0; bit--) {
        int b = (num >> bit) & 1;
        int opposite = 1 - b;

        if (node->child[opposite]) {
            value |= (1 << bit);
            node = node->child[opposite];
        } else {
            node = node->child[b];
        }
    }

    return value;
}

public:
    int findMaximumXOR(vector<int>& nums) {
        root = new Node();

        for (int x : nums) {
            insert(x);
        }

        int answer = 0;

        for (int x : nums) {
            answer = max(answer, getMaxXor(x));
        }

        return answer;
    }
};

```

20.4 Complexity

For n numbers and 32 bits:

Time: $O(32n)$

Space: $O(32n)$

Effectively linear.

21. Full Problem Walkthrough 8 — LeetCode 745: Prefix and Suffix Search

21.1 Problem Summary

Design WordFilter:

```
f(prefix, suffix)
```

Return largest index of word having that prefix and suffix.

21.2 Hard Trie Trick

Insert combined strings:

```
suffix + "#" + word
```

For every suffix of word.

Example:

word = apple

Insert:

```
"apple#apple"  
"pple#apple"  
"ple#apple"  
"le#apple"  
"e#apple"  
"#apple"
```

Then query:

```
suffix + "#" + prefix
```

This converts prefix + suffix query into a prefix search.

21.3 C++ Code

```
class WordFilter {  
private:  
    struct TrieNode {  
        TrieNode* children[27];  
        int weight;  
  
        TrieNode() {  
            memset(children, 0, sizeof(children));  
            weight = -1;  
        }  
    };  
  
    TrieNode* root;  
  
    int getIndex(char c) {  
        if (c == '#') {  
            return 26;  
        }  
    }  
};
```



```

    }

    return c - 'a';
}

void insert(string s, int weight) {
    TrieNode* node = root;

    for (char c : s) {
        int index = getIndex(c);

        if (!node->children[index]) {
            node->children[index] = new TrieNode();
        }

        node = node->children[index];
        node->weight = weight;
    }
}

public:
    WordFilter(vector<string>& words) {
        root = new TrieNode();

        for (int i = 0; i < words.size(); i++) {
            string word = words[i];

            for (int start = 0; start <= word.size(); start++) {
                string combined = word.substr(start) + "#" + word;
                insert(combined, i);
            }
        }
    }

    int f(string pref, string suff) {
        string query = suff + "#" + pref;

        TrieNode* node = root;

        for (char c : query) {
            int index = getIndex(c);

            if (!node->children[index]) {
                return -1;
            }

            node = node->children[index];
        }
    }
}

```

```
        return node->weight;
    }
};
```

21.4 Complexity

If average word length is L:

Construction:

$O(\text{words} * L^2)$

Query:

$O(\text{prefix length} + \text{suffix length})$

Memory is high but accepted for the problem constraints.

22. Full Problem Walkthrough 9 — LeetCode 1032: Stream of Characters

22.1 Problem Summary

Given words.

For each queried character, return true if any suffix of stream equals a word.

22.2 Trie Trick

We need suffix matching.

Insert reversed words into Trie.

Keep stream characters.

For query, scan stream backward through Trie.

22.3 C++ Code

```
class StreamChecker {
private:
    struct TrieNode {
        TrieNode* children[26];
        bool isEnd;

        TrieNode() {
            memset(children, 0, sizeof(children));
            isEnd = false;
        }
    };

    TrieNode* root;
    string stream;
    int maxLen = 0;

    void insertReversed(string word) {
        TrieNode* node = root;

        for (int i = word.size() - 1; i >= 0; i--) {
            int index = word[i] - 'a';
```

```

        if (!node->children[index]) {
            node->children[index] = new TrieNode();
        }

        node = node->children[index];
    }

    node->isEnd = true;
}

public:
    StreamChecker(vector<string>& words) {
        root = new TrieNode();

        for (string word : words) {
            insertReversed(word);
            maxLen = max(maxLen, (int)word.size());
        }
    }

    bool query(char letter) {
        stream.push_back(letter);

        if (stream.size() > maxLen) {
            stream.erase(stream.begin());
        }

        TrieNode* node = root;

        for (int i = stream.size() - 1; i >= 0; i--) {
            int index = stream[i] - 'a';

            if (!node->children[index]) {
                return false;
            }

            node = node->children[index];

            if (node->isEnd) {
                return true;
            }
        }

        return false;
    }
};

```

22.4 Complexity

Per query:

$O(\text{maxWordLength})$

Space:

$O(\text{total word characters})$

23. Full Problem Walkthrough 10 — LeetCode 720: Longest Word in Dictionary

23.1 Problem Summary

Find longest word such that every prefix of it is also in dictionary.

If tie, return lexicographically smallest.

Example:

`["w", "wo", "wor", "worl", "world"]`

Answer:

`"world"`

23.2 Approach

Sort words lexicographically.

Use hash set to check if prefix chain exists.

Trie also works.

We show Trie DFS.

23.3 C++ Trie Code

```
class Solution {
private:
    struct TrieNode {
        TrieNode* children[26];
        bool isEnd;

        TrieNode() {
            memset(children, 0, sizeof(children));
            isEnd = false;
        }
    };

    TrieNode* root;
    string answer;

    void insert(string word) {
        TrieNode* node = root;

        for (char c : word) {
```

```

        int index = c - 'a';

        if (!node->children[index]) {
            node->children[index] = new TrieNode();
        }

        node = node->children[index];
    }

    node->isEnd = true;
}

void dfs(TrieNode* node, string& path) {
    if (path.size() > answer.size()) {
        answer = path;
    }

    for (int i = 0; i < 26; i++) {
        if (node->children[i] && node->children[i]->isEnd) {
            path.push_back('a' + i);

            dfs(node->children[i], path);

            path.pop_back();
        }
    }
}

public:
    string longestWord(vector<string>& words) {
        root = new TrieNode();

        for (string word : words) {
            insert(word);
        }

        string path;
        dfs(root, path);

        return answer;
    }
};

```

23.4 Why DFS in Alphabetical Order?

If same length exists, lexicographically smaller should win.

Because we only update when length is greater, first found lexicographic word of that length remains.

24. Full Problem Walkthrough 11 — LeetCode 820: Short Encoding of Words

24.1 Problem Summary

Given words, encode them into shortest reference string such that every word appears as suffix ending with #.

Example:

```
["time", "me", "bell"]
```

Shortest encoding:

```
"time#bell#"
```

because me is suffix of time.

Answer length:

```
10
```

24.2 Trie Insight

Suffix relationship becomes prefix relationship if words are reversed.

Insert reversed words into Trie.

Only leaf words contribute to encoding length.

24.3 Simpler Hash Set Approach

Remove all suffixes.

```
class Solution {
public:
    int minimumLengthEncoding(vector<string>& words) {
        unordered_set<string> st(words.begin(), words.end());

        for (string word : words) {
            for (int i = 1; i < word.size(); i++) {
                st.erase(word.substr(i));
            }
        }

        int answer = 0;

        for (string word : st) {
            answer += word.size() + 1;
        }

        return answer;
    }
};
```

24.4 Trie Note

Reversed Trie is more structurally aligned, but hash set version is simpler.

Both are useful to know.

25. Full Problem Walkthrough 12 — Count Prefix Pairs

25.1 Problem Summary

Given words, count pairs i, j where $i < j$ and $\text{words}[i]$ is a prefix of $\text{words}[j]$.

25.2 Trie Approach

Insert previous words.

For each new word, walk Trie and count how many word endings appear along its path.

25.3 C++ Code

```
class Solution {
private:
    struct TrieNode {
        TrieNode* children[26];
        int endCount;

        TrieNode() {
            memset(children, 0, sizeof(children));
            endCount = 0;
        }
    };

    TrieNode* root;

    void insert(string word) {
        TrieNode* node = root;

        for (char c : word) {
            int index = c - 'a';

            if (!node->children[index]) {
                node->children[index] = new TrieNode();
            }

            node = node->children[index];
        }

        node->endCount++;
    }

    int countPrefixes(string word) {
        TrieNode* node = root;
        int count = 0;

        for (char c : word) {
```

```

        int index = c - 'a';

        if (!node->children[index]) {
            return count;
        }

        node = node->children[index];
        count += node->endCount;
    }

    return count;
}

public:
    int countPrefixPairs(vector<string>& words) {
        root = new TrieNode();

        int answer = 0;

        for (string word : words) {
            answer += countPrefixes(word);
            insert(word);
        }

        return answer;
    }
};

```

26. Trie Memory Optimization

26.1 Problem

A Trie node with:

```
TrieNode* children[26];
```

uses memory for all 26 pointers, even if most are null.

For many sparse nodes, this is wasteful.

26.2 Options

Option 1 — unordered_map children

```
unordered_map<char, TrieNode*> children;
```

Pros:

- memory efficient for sparse alphabets
- supports arbitrary characters

Cons:

- slower than array
- more overhead per map

Option 2 — vector of nodes with integer indices


```
struct Node {
    int child[26];
    bool isEnd;
};
```

Store all nodes in vector.

This avoids many small heap allocations.

[Option 3 — compressed Trie / radix tree](#)

Compress chains of single-child nodes.

Useful for large dictionaries.

27. Vector-Based Trie Implementation

This is often faster and cleaner in contests.

```
class Trie {
private:
    struct Node {
        int child[26];
        bool isEnd;

        Node() {
            memset(child, -1, sizeof(child));
            isEnd = false;
        }
    };

    vector<Node> trie;

public:
    Trie() {
        trie.push_back(Node());
    }

    void insert(string word) {
        int node = 0;

        for (char c : word) {
            int index = c - 'a';

            if (trie[node].child[index] == -1) {
                trie[node].child[index] = trie.size();
                trie.push_back(Node());
            }

            node = trie[node].child[index];
        }
    }
};
```

```

        trie[node].isEnd = true;
    }

    bool search(string word) {
        int node = 0;

        for (char c : word) {
            int index = c - 'a';

            if (trie[node].child[index] == -1) {
                return false;
            }

            node = trie[node].child[index];
        }

        return trie[node].isEnd;
    }
};

```

28. Common Trie Edge Cases

28.1 Empty String

Decide whether empty string is valid word.

If yes, root may need:

```
root->isEnd = true;
```

28.2 Prefix Is Also Word

Example:

```
app
apple
```

Need isEnd.

28.3 Duplicate Insertions

If duplicates matter, store count instead of bool.

```
int endCount;
```

28.4 Uppercase / Non-Lowercase

Array size 26 only works for lowercase English letters.

Use map for general characters.

28.5 Memory Leak

In interview code, usually ignored.

In production, use smart pointers or destructor.

29. Common Trie Mistakes

Mistake 1 — Forgetting isEnd

Then prefix search and word search become confused.

Mistake 2 — Returning true After Walking Word

For search, must return:

```
node->isEnd
```

not just path exists.

Mistake 3 — Treating Prefix as Word

If only apple inserted, searching app should return false unless app inserted.

Mistake 4 — Wildcard DFS Not Checking Null Node

Always check:

```
if (!node) return false;
```

Mistake 5 — Word Search II Duplicate Results

Set node->word = "" after finding.

Mistake 6 — Binary Trie Bit Range

Use correct bit range.

For non-negative int, 31 to 0 is common.

30. Interview Explanation Templates

30.1 Basic Trie

A Trie stores words character by character. Words with the same prefix share nodes, so insertion and search take time proportional to word length.

30.2 startsWith

To check a prefix, I walk the Trie according to the prefix characters. If the path exists, the prefix exists.

30.3 WordDictionary Wildcard

For a normal character, I follow one child. For a wildcard, I recursively try all possible children.

30.4 Word Search II

I insert all words into a Trie and then DFS on the board. The Trie lets me stop early whenever the current path is not a prefix of any word.

30.5 Binary Trie XOR

To maximize XOR, for every bit I try to move to the opposite bit in the Trie, because opposite bits produce 1 in XOR.

31. Complexity Patterns

Let:

```
L = word length  
N = number of words  
S = total characters
```

Operation / Problem	Time	Space
Insert word	$O(L)$	$O(\text{new nodes})$
Search word	$O(L)$	$O(1)$ extra
startsWith	$O(L)$	$O(1)$ extra
Wildcard search	worst $O(26^L)$	recursion
Replace Words	$O(\text{dictionary chars} + \text{sentence chars})$	$O(\text{dictionary chars})$
Word Search II	high worst-case, heavily pruned	$O(\text{total word chars})$
Search Suggestions	$O(\text{sort} + \text{total chars})$	$O(\text{total chars})$
MapSum insert/sum	$O(L)$	$O(\text{total chars})$
Binary Trie maximum XOR	$O(32N)$	$O(32N)$
Prefix/Suffix Search	$O(NL^2)$ build	$O(\text{query length})$

32. Expanded Practice Set

32.1 Must-Do Medium

- 26. Implement Trie
- 27. Design Add and Search Words Data Structure
- 28. Replace Words
- 29. Map Sum Pairs
- 30. Search Suggestions System
- 31. Longest Word in Dictionary
- 32. Short Encoding of Words

32.2 Strong Interview Practice

- 33. Word Search II
- 34. Maximum XOR of Two Numbers in an Array
- 35. Prefix and Suffix Search
- 36. Stream of Characters
- 37. Count Prefix Pairs
- 38. Design Search Autocomplete System
- 39. Camelcase Matching

32.3 Hard / Stretch

- 40. Palindrome Pairs
- 41. Concatenated Words
- 42. Word Squares
- 43. Maximum Genetic Difference Query
- 44. Count Prefix and Suffix Pairs II

33. Suggested Study Order

Step 1 — Basic Trie

Solve:

- 47. Implement Trie
- 48. Map Sum Pairs

Goal:

Learn node structure and prefix traversal.

Step 2 — Wildcard Trie

Solve:

- 49. Add and Search Word

Goal:

Learn DFS branching.

Step 3 — Dictionary Replacement / Suggestions

Solve:

- 50. Replace Words
- 51. Search Suggestions System
- 52. Longest Word in Dictionary

Goal:

Learn practical prefix applications.

Step 4 — Trie + Backtracking

Solve:

- 53. Word Search II

Goal:

Master Trie pruning with DFS.

Step 5 — Binary Trie

Solve:

- 54. Maximum XOR

Goal:

Understand Trie beyond strings.

Step 6 — Advanced Trie Design

Solve:

- 55. Prefix and Suffix Search
- 56. Stream of Characters
- 57. Palindrome Pairs

Goal:

Learn reversed Trie and combined-key tricks.

34. Pattern Decision Table

Problem Clue	Trie Strategy
insert/search/prefix	basic Trie

Problem Clue	Trie Strategy
. wildcard	DFS over children
board + dictionary	Trie + DFS backtracking
shortest root prefix	stop at first isEnd
autocomplete top 3	store suggestions at nodes
prefix sum of values	store prefix sum in nodes
maximum XOR	binary Trie
suffix query	reversed Trie
prefix + suffix query	suffix + "#" + word trick
stream suffix matching	reversed Trie + backward scan
every prefix must exist	DFS only through isEnd nodes

35. Mini Problem 1 — Count Words Equal To and Starting With

Design Trie with counts.

```

class Trie {
private:
    struct Node {
        Node* child[26];
        int prefixCount;
        int endCount;

        Node() {
            memset(child, 0, sizeof(child));
            prefixCount = 0;
            endCount = 0;
        }
    };

    Node* root;

public:
    Trie() {
        root = new Node();
    }

    void insert(string word) {

```

```

Node* node = root;

for (char c : word) {
    int index = c - 'a';

    if (!node->child[index]) {
        node->child[index] = new Node();
    }

    node = node->child[index];
    node->prefixCount++;
}

node->endCount++;
}

int countWordsEqualTo(string word) {
    Node* node = root;

    for (char c : word) {
        int index = c - 'a';

        if (!node->child[index]) {
            return 0;
        }

        node = node->child[index];
    }

    return node->endCount;
}

int countWordsStartingWith(string prefix) {
    Node* node = root;

    for (char c : prefix) {
        int index = c - 'a';

        if (!node->child[index]) {
            return 0;
        }

        node = node->child[index];
    }

    return node->prefixCount;
}
};

```

36. Mini Problem 2 — Delete Word From Trie

For count-based Trie:

```
void erase(string word) {
    Node* node = root;

    for (char c : word) {
        int index = c - 'a';

        node = node->child[index];
        node->prefixCount--;
    }

    node->endCount--;
}
```

This assumes word exists.

For production, check existence first.

37. Mini Problem 3 — Check If Any Word Has Prefix

```
bool hasPrefix(string prefix) {
    TrieNode* node = root;

    for (char c : prefix) {
        int index = c - 'a';

        if (!node->children[index]) {
            return false;
        }

        node = node->children[index];
    }

    return true;
}
```

38. Mini Problem 4 — Binary Trie Minimum XOR

For minimum XOR, prefer same bit first instead of opposite bit.


```

int getMinXor(int num) {
    Node* node = root;
    int value = 0;

    for (int bit = 31; bit >= 0; bit--) {
        int b = (num >> bit) & 1;

        if (node->child[b]) {
            node = node->child[b];
        } else {
            value |= (1 << bit);
            node = node->child[1 - b];
        }
    }

    return value;
}

```

39. Common Edge Case Checklist

Before submitting Trie code, test:

```

insert "app", search "app"
insert "apple", search "app"
startsWith "app"
duplicate word insertion
empty dictionary
word not found
prefix not found
wildcard at beginning
wildcard at end
word search duplicate result
uppercase / invalid characters
maximum XOR with one number
prefix-suffix query no match

```

40. Debugging Trie Problems

If Trie code fails, check:

58. Did you initialize all children to null?
59. Did you set isEnd at the end of insert?
60. Are you returning isEnd for exact search?
61. Are you returning true for prefix only in startsWith?
62. Is character indexing correct?
63. Are duplicate words handled correctly?
64. Is wildcard DFS trying all children?
65. Are board cells restored after DFS?
66. Are duplicate Word Search II results removed?

41. What to Master Before Chapter 47

Before moving to Rolling Hash / Rabin-Karp, make sure you can:

- Implement Trie from scratch
 - Explain Trie node structure
 - Solve Implement Trie
 - Solve Add and Search Word
 - Solve Replace Words
 - Solve Search Suggestions System
 - Solve Map Sum Pairs
 - Solve Word Search II
 - Solve Maximum XOR using binary Trie
 - Understand prefix/suffix tricks
 - Understand reversed Trie
 - Understand Trie memory tradeoffs
-

42. Final Cheat Sheet

Insert

```
for (char c : word) {
    int index = c - 'a';

    if (!node->children[index]) {
        node->children[index] = new TrieNode();
    }

    node = node->children[index];
}

node->isEnd = true;
```

Search

```
for (char c : word) {
    int index = c - 'a';

    if (!node->children[index]) {
        return false;
    }

    node = node->children[index];
}

return node->isEnd;
```

startsWith

```
return pathExists(prefix);
```

Wildcard

```
if (c == '.') {  
    try all children;  
}
```

Word Search II

```
if (!node->children[index]) {  
    return;  
}
```

Binary Trie XOR

```
opposite = 1 - bit;  
  
if (node->child[opposite]) {  
    answer |= (1 << bit);  
    node = node->child[opposite];  
}
```

Prefix Sum Trie

```
delta = newValue - oldValue;  
node->sum += delta;
```

43. Core Takeaway

Trie is the data structure of prefixes.

Use it when many strings share prefixes or when prefix-based queries are central.

The core operations are:

```
insert  
search  
startsWith
```

From these, advanced problems build:

- wildcard search with DFS
- grid word search with backtracking
- autocomplete with suggestions
- prefix sums with node values
- suffix search with reversed words
- maximum XOR with binary Trie

The most important implementation details are:

- initialize children
- mark word endings
- distinguish word search from prefix search
- prune early
- handle memory carefully

Once you master basic Trie, many hard-looking string design problems become much easier.

